JORDAN ANIUZU COMPUTER VISION REPORT FOR ASSIGNMENT 2 (B00151878)

The aim of this assignment was to build a deep learning model that classifies chest X-ray images into three specific categories shown in the report being: “Bacterial Pneumonia, Normal, and Viral Pneumonia”.

The model was built **using “TensorFlow and Keras”** using transfer learning with **“MobileNetV2”.** The final model had **a 73.7%** accuracy on the test set after 49 minutes of training on CPU.

Q1: How long did the network take to train?

Total training time was **49.1 minutes on CPU**. Training ran in two phases. **Phase 1 (aka 30 mins):** Trained only the new classification layers on top of MobileNetV2 while keeping the pretrained weights frozen.

Phase 2 (19 mins), Unfroze the top 30 layers of MobileNetV2 and fine-tuned at a low learning rate to adapt to X-ray images. (It ran on CPU only because TensorFlow 2.11+ dropped native GPU support on Windows. A GPU would have cut this down to under 5 minutes)

Q2: Is the dataset balanced? What is the distribution? If not can I do anything to address this?

From the data received from the test, it is not balanced. The training set breaks down like this:

BACTERIAL	2,596	48%
NORMAL	1,461	27%
VIRAL	1,363	25%

Bacterial has almost double the amount of samples of the other two classes. In order to amend this, I had used class weights during training so the model had been adjusted more for getting the minority classes wrong. In this case, VIRAL got a weight of 1.33 vs BACTERIAL at 0.70.

Q3: Is the network overfitting? Why or why not? If so what can I do to address this, see GlobalAveragePooling2D ?

In this instance, yes. Some overfitting showed up in Phase 2. The validation loss kept going up while training loss went down, creating a sharp contrast signalling possible overfitting. Early Stopping caught it and restored the best weights automatically before it got too bad.

To reduce overfitting the model used Dropout (0.4 and 0.3), Batch Normalisation, Data Augmentation, and GlobalAveragePooling2D instead of Flatten to keep the parameter count down.

Q4: Can you perform any data augmentation?

In this case we can do so. Augmentation was applied to all the given training images. The following random transforms were used:

Augmentation	Setting
Horizontal Flip	Random
Rotation	+/- 10 degrees
Zoom	+/- 10%
Translation	+/- 10% x and y

Augmentation was only applied during training and not validation or testing. This helps/helped the model generalize by making sure it never sees the exact same image twice.

Q5: What layers are in the network? Can I alter anything here to improve matters? Perhaps take a look at the keras tuner

Input	(224, 224, 3)	Takes in the X-ray image
MobileNetV2	(7, 7, 1280)	Pretrained feature extractor
GlobalAveragePooling2D	(1280)	Squashes spatial dimensions down
Dense 256 + ReLU	(256)	Learns class-specific features
BatchNorm + Dropout 0.4	(256)	Reduces overfitting
Dense 128 + ReLU	(128)	Refines features further
Dropout 0.3	(128)	More regularisation
Dense 3 + Softmax	(3)	Outputs class probabilities

The total number of parameters was 2.6 million. In this case, only 361k were trainable in Phase 1 which is very low compared to the original number of parameters and this could be improved by using a larger backbone getting swapped like EfficentNetB3 or if not, cutting the dense layers is another solution.

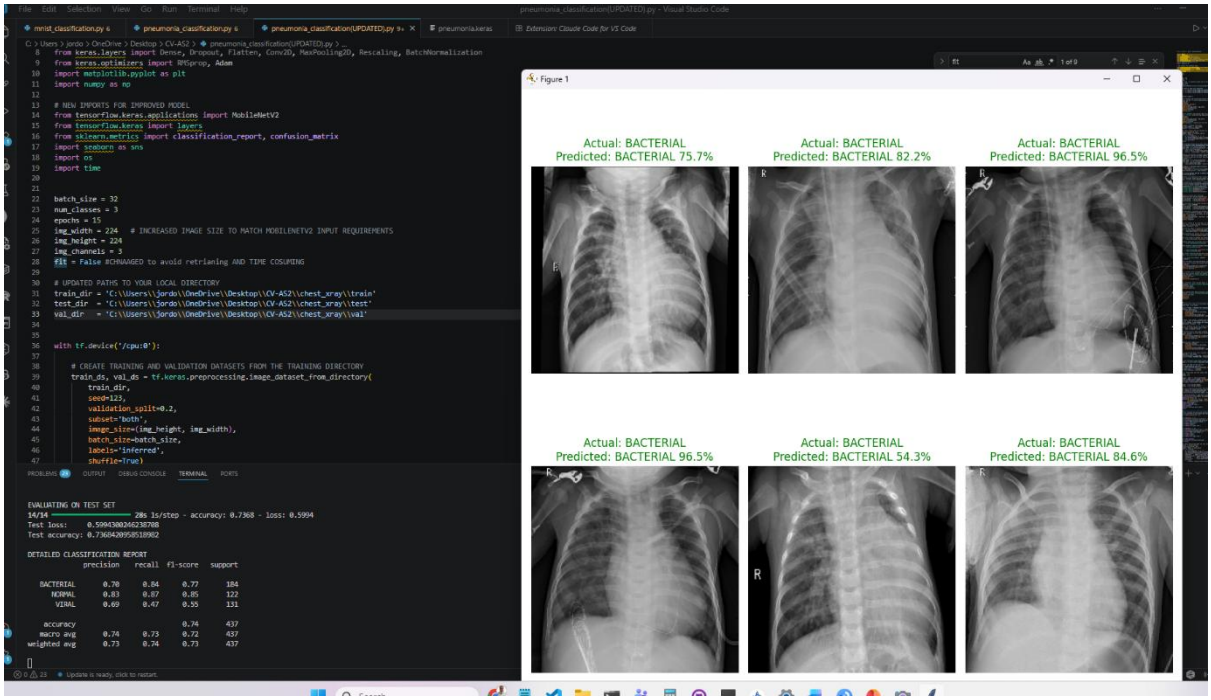
Q6: Is transfer learning an option here using a pre-trained model?

In this case it was and it was aso the main improvement made here. MobileNetV2 pretrained on ImageNet was used as the backbone. The idea here was that features learned from millions of natural images such as edges or shapes also apply to medical images meaning there is TECHNCIALLY no need to start from nothing.

Phase 1 froze the base and trained only the new head whilst phase 2 unfroze the top 30 layers and fine-tuned at a learning rate of 1e-5 so the pretrained weights were not destroyed. This approach consistently outperforms training a CNN from scratch on datasets this size.

Q7: What are the per class precision, recall and F1 scores? What do they mean and which is best here.

Below is a table representation of all the data asked for



Class	Precision	Recal	F1
BACTERIAL	0.70	0.84	0.77
NORMAL	0.83	0.87	0.85
VIRAL	0.69	0.47	0.55

Overall	0.74	0.73	0.72

<u>Meanings of Per class Precision, recall and F1:</u>

- **Precision** = of everything the model labelled as X, how many were **actually X**.
- **Recall** = of everything that was actually X, **how many did the model catch.**
- **F1** is the balance between both.
- **NORMAL** has the best recall at 0.87.

Q8: How might you make the model better at finding the sick patients?

To make it better for finding sick patients, we can look to increase the **class weight for VIRAL** even further to force the model to focus on those cases whilst lowering the classification threshold for pneumonia classes so the model flags more positives, trading precision for recall.

In other instances we can use **Focal Loss** instead of standard cross-entropy, which focuses training effort on hard-to-classify examples whilst collect more viral pneumonia training data since it is the most underrepresented and weakest class.

Q9: CIs it possible to see what the CNN is seeing? Can I use tf-explain (e.g.GradCam) to see this?

Through using GradCAM we can produces a heatmap over the image showing which parts most influenced the prediction. For chest X-rays this is useful to check the model is looking at the lungs and not background noise or scan artefacts.

In this case, GradCAM works by computing the gradient of the predicted class score with respect to the last convolutional layer, then overlaying that as a colour heatmap on the original image. I had not implemented it due to time constraints but the tf-explain library makes it straightforward to add.

Q10: Anything else that you come across in your research to improve the model.

If I were to further improve the model, I would most likely swap **MobileNetV2 for EfficientNetB**3 which tends to performs better on image classification benchmarks. Id also utilize UseTest Time Augmentation which can run augmentation at inference time and average predictions across multiple versions of each test image for a free accuracy boost.

I'd also try Mixup augmentation in which blends two training images together to create smoother decision boundaries whilst running k-fold cross-validation instead of a single split for a more reliable accuracy estimate.

Overall Conclusion

The overall conclusion was that the model achieved 73.7% test accuracy using MobileNetV2 transfer learning, class weighting, data augmentation, and a two-phase training strategy whilst the VIRAL class was the hardest to classify. This does makes sense due to the similarity of resemblance between viral and bacterial pneumonia on X-rays, however, with GPU access and more training time, further improvements through **EfficientNet or GradCAM visualization** would be a lot more straightforward and streamlined for the next steps.